MODULE 5

Graphical Models

Throughout this book we have seen that machine learning brings together computer science and statistics. Nowhere is this more clearly shown than in one of the most popular areas of current research in machine learning: graphical models (or more completely, probabilistic graphical models), which use graph theory with all its underlying computational and mathematical machinery in order to explain probabilistic models.

The graphs used in graphical models are the exact ones that are taught in basic algorithms classes: a set of nodes, together with links between them, which can be either directed (i.e., have arrows on them so that you can only go one way along them) or not. There are two basic types of graphical models, depending upon whether or not the edges are directed. We will focus primarily on directed graphs, but the undirected kind (known as Markov Random Fields) are described in Section 16.2. For such a simple data structure, graphs have turned out to be incredibly powerful in many different parts of computer science, from constructing compilers to managing computer networks. For this reason, there are lots of readily available algorithms for finding shortest paths (Floyd's and Djiksta's algorithms, which we've already discussed briefly in Section 6.6), determining cycles, etc. Any good book on algorithms will give details of these and many other graph algorithms.

For our part, we are interested in using graphs to encode probability distributions and so we need to decide what nodes and links are in this context. The nodes are fairly obvious. We generate a node for each random variable, and label it accordingly. In this book, we will only consider discrete variables, so that there is a finite number of possible values that the random variable can take. Given a continuous variable we will discretise it into a finite set. While this loses information, it makes the problem much simpler. The alternative is to specify the variable by a probability density function, which can be done, but makes the whole thing harder to describe and understand.

The question is what to make the links represent. Perhaps the best way to think about this is to ask what it means if two nodes are not linked. In this case we are saying that there is no connection between those two variables, which is the same as saying that they are independent. Except it isn't quite as simple as that, because two nodes could be linked through a third node. Have a look at the right of Figure 16.1, where C is not directly linked to B, but there is a link through A. For this reason we have to be careful and talk about conditional independence: C is conditionally independent of B, given A.

We use directed links because these relationships are not symmetrical (unless the variables are independent, in which case there is no link). What does the simplest connected graph that we can make, the one on the left of Figure 16.1, mean? There is a rather loose interpretation of the link, which is to say that 'A' causes 'B' (but note that this isn't quite the same semantic usage that we normally have for 'causes', since there may be several

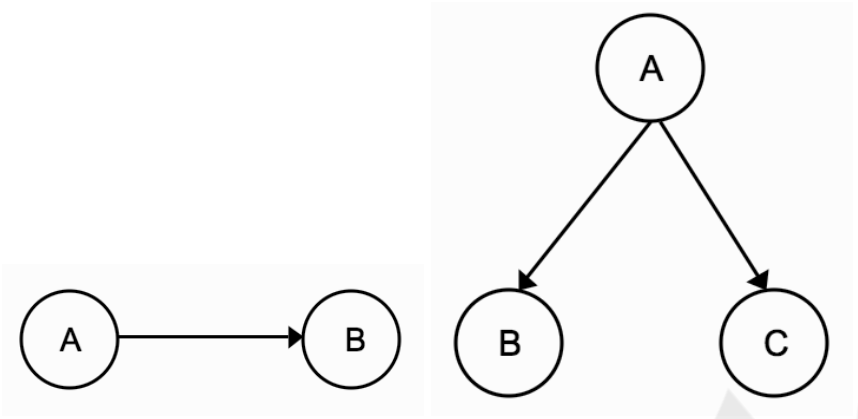


FIGURE 16.1 Two simple graphical models. The arrows denote causal relationships between nodes that represent features.

variables that are all involved in causing B). This is a useful intuition to have, but it is not really correct. More properly, the graph tells us that the probability of A and B is the same as the probability of A times the probability of B conditioned on A: $P(a, b) = P(b|a)P(a)$. If there is no direct link between two nodes then they are conditionally independent of each other.

There is a third thing that we need in order to specify the problem properly, which is the conditional probability table for each variable. This specifies what the probabilities are for each of the nodes, conditioned on any nodes that are its parents.

If we wanted to work out a value for $P(a, b)$, then we would need a distribution table for $P(a)$ and one for $P(b|a)$. The nodes are separated into those where we can see their values directly—observed nodes—and hidden or latent nodes, whose values we hope to infer, and which may not have clear meanings in all cases.

The basic concept of the graphical model is very simple, which makes it all the more amazing that it produces a powerful set of tools for understanding and creating machine learning algorithms. We will start by looking at the most general model, the Bayesian Belief Network or more simply, Bayesian Network, and see how they are represented, and the difficulties involved in dealing with them. Following this, we will identify a few places where these difficulties can be overcome, resulting in some very important algorithms that solve a variety of different tasks. In particular, we will look at Markov Random Fields (MRFs), Hidden Markov Models (HMMs), the Kalman Filter, and particle filter.

16.1 BAYESIAN NETWORKS

To start with, we will consider directed graphs, and make one restriction to them, namely that they must not contain cycles, that is, there cannot be any loops in the graphs. These graphs go by the rather unlovely name of DAGs: directed, acyclic graphs, but for graphical models, when they are paired with the conditional probability tables, they are called Bayesian networks. In order to see what we can do with such a network, we need an example.

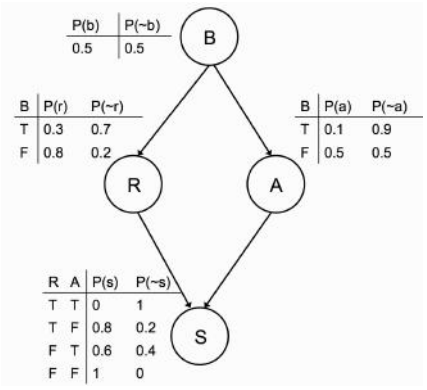


FIGURE 16.2 The sample graphical model. 'B' denotes a node stating whether the exam was boring, 'R' whether or not you revised, 'A' whether or not you attended lectures, and 'S' whether or not you will be scared before the exam.

16.1.1 Example: Exam Fear

Figure 16.2 shows a graph with a full set of distribution tables specified. It is a handy guide to whether or not you will be scared before an exam based on whether or not the course was boring ('B'), which was the key factor you used to decide whether or not to attend lectures ('A') and revise ('R'). We can use it to perform inference in order to decide the likelihood of you being scared before the exam ('S'). There are two kinds of inferences, depending on whether the observations that are made come from the top of the graph or the bottom. If we have a set of observations that can be used to predict an unknown outcome, then we are doing top-down inference or prediction, whereas if the outcome is known, but the causes are hidden, then we are doing bottom-up inference or diagnosis. Either way, we are working out the values of the hidden (unknown) nodes given information about the observed nodes. For the example in Figure 16.2 we will start by predicting whether or not you will be scared before the exam, so it is the outcome that is hidden.

In order to compute the probability of being scared, we need to compute $P(b, r, a, s)$, where the lower-case letters indicate particular values that the upper-case variables can take. The wonderful thing about the graphical model is that we can read the conditional probabilities from the graph—if there is no direct link, then variables are conditionally independent given a node that is already included, so those variables are not needed. For this reason, the computation we need for Figure 16.2 is:

$$\begin{aligned} P(s) &= \sum_{b,r,a} P(b, r, a, s) \\ &= \sum_{b,r,a} P(b) \times P(r|b) \times P(a|b) \times P(s|r, a) \\ &= \sum_{b} P(b) \times \sum_{r,a} P(r|b) \times P(a|b) \times P(s|r, a). \end{aligned} \tag{16.1}$$

If we know particular values for the three observable nodes, then we can plug them in and work out the probability. In fact, the conditional independence gives us even more: if I know both whether or not you attended lectures and whether or not you revised, then I don't

need to know if the course was boring, since there is no direct connection between 'B' and 'S'. Suppose that you didn't attend lectures, but did revise. In that case, the probability of you being scared can be read off the final distribution table as 0.8. The power of the graphical model is when you don't have full information. It is possible to marginalise over any of those variables by summing up the values. So suppose that you know that the course was boring, and want to work out how likely it is that you will be scared before the exam. In that case you can ignore the $P(b)$ terms, and just need to sum up the probabilities for r and a using Equation (16.1):

$$\begin{aligned} P(s) &= 0.3 \times 0.1 \times 0 + 0.3 \times 0.9 \times 0.8 + 0.7 \times 0.1 \times 0.6 + 0.7 \times 0.9 \times 1 \\ &= 0.328. \end{aligned} \tag{16.2}$$

The backwards inference, or diagnosis, can also be useful. Suppose that I see you looking very scared outside the exam. You look vaguely familiar, but I'm not sure whether or not you came to the lectures. I might want to work out why you are scared—was it because you didn't come to the lectures, or because you didn't revise? To perform this calculation I need to use Bayes' rule to turn the conditional probabilities around, just as was done for the Bayes' classifier in Chapter 7. So the computations that I need are (where $P(s)$ is the normalising constant found by summing over all values of r, a, and b, i.e., Equation (16.2)):

$$\begin{aligned} P(r|s) &= \frac{P(s|r)P(r)}{P(s)} \\ &= \frac{\sum_{b,a} P(b,a,r,s)}{P(s)} \\ &= \frac{0.5 \cdot (0.3 \cdot 0.1 \cdot 0 + 0.3 \cdot 0.9 \cdot 0.8) + 0.5 \cdot (0.8 \cdot 0.5 \cdot 0 + 0.8 \cdot 0.5 \cdot 0.8)}{P(s)} \\ &= \frac{0.268}{0.684} = 0.3918. \end{aligned} \tag{16.3}$$

$$\begin{aligned} P(a|s) &= \frac{P(s|a)P(a)}{P(s)} \\ &= \frac{0.144}{0.684} = 0.2105. \end{aligned} \tag{16.4}$$

This use of Bayes' rule is the reason why this type of graphical model is known as a Bayesian network. Even in this very simple example, the inference was not trivial, since there were a lot of calculations to do. However, the problem is actually rather worse than that. The computational cost of the simple algorithm we used (start at the root, and follow each link through the graph to perform the computation) is $\mathcal{O}(2^N)$ for a graph with N nodes where each node can be either true or false. In general the problem of exact inference on Bayesian networks is NP-hard (technically, it is actually #P-hard, which is even worse). However, for so-called polytrees where there is at most one path between any two nodes, the computational cost is much smaller—linear in the size of the network.

Unfortunately, it is rare to find such polytrees in real examples, so we can either try to turn other networks into polytrees, or consider only approximate inference, which is the most common solution to the problem, and the method that we'll consider next. We can speed things up a little by getting things into the form of Equation (16.1), where the summations were carefully placed as far to the right as possible, so that program loops can be minimised. By doing this the algorithm is as efficient as possible, but it is still NP-hard.

This is sometimes known as the variable elimination algorithm, which is a variation on the bucket elimination algorithm. The idea is to convert the conditional probability tables into what are called λ tables, which simply list all of the possible values for all variables, and which initially contain the conditional probabilities. For example, the λ table for the 'S' variable in Figure 16.2 is:

R	A	S	λ
T	T	T	0
T	T	F	1
T	F	T	0.8
T	F	F	0.2
F	T	T	0.6
F	T	F	0.4
F	F	T	1
F	F	F	0

If I see you looking scared outside the exam (so that S is true), then I can eliminate it from the graph by removing from each table all rows that have S false in them, and deleting the S column. This simplifies things a little, but I have to do rather more in order to compute the probability of you having attended lectures. I don't know whether you revised or not, and I don't know if you found the lectures boring, so I have to marginalise over these variables. The order in which we marginalise doesn't change the correctness (although more advanced algorithms can improve the speed by taking advantage of conditional independence) so we'll pick R first. To eliminate it from the graph, we have to find all of the λ tables that contain it (there will be two of them containing R: the one for R itself and the one that we have just modified to remove S). To remove R, we have to add together the products of the λ values that correspond to places where the other values match. So to complete the entry where B is true and A is false, we have to multiply together the values where B, A, R are respectively true, false, true in the two tables and then add to that the product of where B, A, R are respectively true, false, false. In other words:

$$\left(\begin{array}{cc|c} B & R & \lambda \\ T & T & 0.3 \\ T & F & 0.7 \\ F & T & 0.8 \\ F & F & 0.2 \end{array}\right) \times \left(\begin{array}{cc|c} R & A & \lambda \\ T & T & 0 \\ T & F & 0.8 \\ F & T & 0.6 \\ F & F & 1 \end{array}\right) \Rightarrow \left(\begin{array}{cc|c} B & A & \lambda \\ T & T & 0.3 \cdot 0 + 0.7 \cdot 0.6 = 0.42 \\ T & F & 0.3 \cdot 0.8 + 0.7 \cdot 1 = 0.94 \\ F & T & 0.8 \cdot 0 + 0.2 \cdot 0.6 = 0.12 \\ F & F & 0.8 \cdot 0.8 + 0.2 \cdot 1 = 0.84 \end{array}\right) \quad (16.5)$$

We can do the same thing in order to eliminate B, which involves all three of the tables, and this will enable the computation of the conditional probability of you attending lectures given that I saw you looking scared before the exam. The benefit of doing things this way is that the whole thing can be written as a general algorithm:

The Variable Elimination Algorithm

- Create the λ tables:
  - for each variable v:
    - make a new table
    - for all possible true assignments x of the parent variables:
      - add rows for $P(v|x)$ and $1 - P(v|x)$ to the table
    - add this table to the set of tables

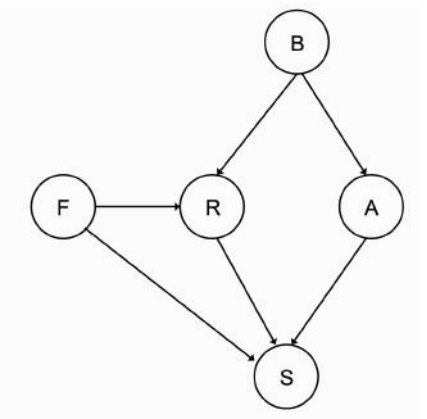


FIGURE 16.3 Adding just one extra node ('F', information about whether or not this is your final year) makes the conditional probability tables significantly more complicated.

- Eliminate known variables v:
  - for each table:
    - remove rows where v is incorrect
    - remove column for v from table
- Eliminate other variables (where x is the variable to keep):
  - for each variable v to be eliminated:
    - create a new table t'
    - for each table t containing v:
      - $v_{\text{true},t} = v_{\text{true},t} \times P(v|x)$
      - $v_{\text{false},t} = v_{\text{false},t} \times P(\neg v|x)$
    - $v_{\text{true},t'} = \sum_t (v_{\text{true},t})$
    - $v_{\text{false},t'} = \sum_t (v_{\text{false},t})$
  - replace tables t with the new one t'
- Calculate conditional probability:
  - for each table:
    - $x_{\text{true}} = x_{\text{true}} \times P(x)$
    - $x_{\text{false}} = x_{\text{false}} \times P(\neg x)$
    - probability is $x_{\text{true}}/(x_{\text{true}} + x_{\text{false}})$

To see that these algorithms do not scale well, consider Figure 16.3, which shows a very simple development of the example in Figure 16.2 by adding just one extra node to the network: whether or not this is your final year ('F'). This makes the network significantly more complicated, since we need another table and extra entries in two of the other tables, and therefore the variable elimination algorithm will take rather longer to run.

Vericircle

16.1.2 Approximate Inference

Since the variable elimination algorithm will only take you so far, for reasonably sized Bayesian networks there is no choice but to perform approximate inference. Fortunately, we have already seen a set of algorithms that are ideally suited to the problem: the Markov Chain Monte Carlo methods that we saw in Chapter 15. There are two other methods of doing approximate inference (loopy belief propagation and mean field approximation), but we will not consider them further; there are references to descriptions of these methods at the end of the chapter.

The basic idea of using MCMC methods in Bayesian networks is to sample from the hidden variables, and then (depending upon the MCMC algorithm employed) weight the samples by their likelihoods. Creating the samples is very easy: for prediction, we start at the top of the graph and sample from each of the known probability distributions. Using Figure 16.2 again, we generate a sample from $P(b)$, and then use that value in the conditional probability tables for 'R' and 'A' to compute $P(r|b = \text{sample value})$ and $P(a|b = \text{sample value})$. These three values are then used to sample from $P(p|b, a, r)$. We can take as many samples as we like in this way, and expect that as the number of samples gets large, so the frequency of specific samples will converge to their expected values.

In this sampling method, we have to work through the graph from top to bottom and select rows from the conditional probability table that match the previous case. This is not what we would do if we were constructing the table by hand. Suppose that you wanted to know how many courses you did not attend the lectures for because the course was boring. You would simply look back through your courses and count the number of boring courses where you didn't go to lectures, ignoring all the interesting courses. We can use exactly this idea if we use rejection sampling (see Section 15.3). The method samples from the unconditional distribution and simply rejects any samples that don't have the correct prior probability. It means that we can sample from each distribution independently, and then throw away any samples that don't match the other variables. This is obviously computationally easier, but we might have to reject a lot of samples.

The solution to this problem is to work out what evidence we already have and use this evidence to assign likelihoods to the other variables that are sampled. Suppose that we sample from $P(b)$ and get value 'true'. If we already know that we did revise, then we weight the observation $P(r|b)$ by the appropriate probability, which is 0.3. We continue through the other variables, sampling where there is no evidence and using the tables to find the probability if we do have evidence. However, we can do rather better than this by using the full MCMC framework. We start by setting values for all of the possible probabilities, based on either evidence or random choices. This gives us an initial state for a Markov chain. Now Gibbs sampling (Section 15.4.4) will find us the maxima of our probability distribution given enough samples.

The probabilities in the network are:

$$p(x) = \prod_j p(x_j|x_{\alpha j}), \tag{16.6}$$

where $x_{\alpha j}$ are the parent nodes of x_j. In a Bayesian network, any given variable is independent of any node that is not their child, given their parents. So we can write:

$$p(x_j|x_{-j}) = p(x_j|x_{\alpha j}) \prod_{k \in \beta(j)} p(x_k|x_{\alpha(k)}), \tag{16.7}$$

where $\beta(j)$ is the set of children of node x_j and x_{-j} signifies all values of x_i except x_j. For

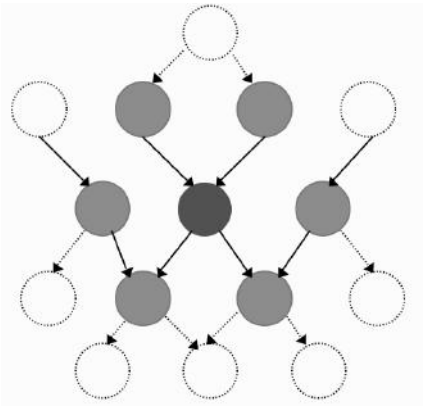


FIGURE 16.4 The Markov blanket of a node is the set of nodes (shaded light grey) that are either parents or children of the node, or other parents of its children (shaded dark grey).

any node we only need to consider its parents, its children, and the other parents of the children, as shown in Figure 16.4. This set is known as the Markov blanket of a node.

Given these calculations, computing the inference on any real Bayesian network generally consists of using Gibbs sampling in order to approximate the inference. For the exam fear example, the algorithm to perform the Gibbs sampling consists of computing the probability distributions (possibly by using parts of the variable elimination algorithm) and then sampling from it:

```
for i in range(nsamples):
    # values contain current samples of b, r, a, s
    values = np.where(np.random.rand(4)<0.5,0,1)
    for j in range(nsteps):
        values=pb_ras(values)
        values=pr_bas(values)
        values=pa_brp(values)
        values=ps_bra(values)
    distribution[values[0]+2*values[1]+4*values[2]+8*values[3]] += 1
distribution /= nsamples
```

For the example, a sample distribution (based on 500 samples, with 10 iterations of each chain) is:

```
b r a s:   dist
1 1 1 1    0.0
1 1 1 0    0.086
1 1 0 1    0.038
1 1 0 0    0.052
1 0 1 1    0.048
```

```
1 0 1 0    0.116
1 0 0 1    0.274
1 0 0 0    0.0
0 1 1 1    0.0
0 1 1 0    0.088
0 1 0 1    0.068
0 1 0 0    0.114
0 0 1 1    0.03
0 0 1 0    0.076
0 0 0 1    0.01
0 0 0 0    0.0
```

16.1.3 Making Bayesian Networks

If we are given the structure and conditional probability tables of the Bayesian network, then we can perform inference on it by using Gibbs sampling or, if the network is simple enough, exactly. However, this raises the important question about where the Bayesian network itself comes from. Unfortunately, the news in this area isn't particularly good: the computational costs of searching over trees are immense, as we shall see. It is not uncommon for people to create the entire network by hand, and only then to use algorithms in order to perform inference on the network. Constructing Bayesian networks by hand is obviously very boring to do, and unless it is based on real data, then it is subjective: putting a whole lot of effort into inference is a waste of time if the data you are inferring about bears no resemblance to reality!

So why is it so difficult to construct Bayesian networks? First, we have already seen that the problem of exact inference on Bayesian networks was NP-hard, which is why we had to use approximate inference. Now let's think about the structure of the graph a little. If there are N nodes (i.e., N random variables in the graph), then how many different graphs are there? For just three nodes ('A', 'B', 'C') we can leave the three unconnected, connect 'A' to 'B' and leave 'C' alone, connect 'B' to 'A' and leave 'C' alone (remember that the links are directional) and lots of variations of that, so that there are seven possible graphs before we have even connected all three nodes to each other. For ten nodes there are $\mathcal{O}(10^{18})$ possible graphs, so we are not going to be searching over all of them. Further, we might want our algorithm to be able to include latent variables, i.e., hidden nodes, which might be a sensible thing to do in terms of explaining the data, but it does make the problem of search even worse.

We've talked about search before: Chapters 9 and 10 are full of search methods. So can we use those methods to solve this problem? The answer is a cautious yes, once we have worked out an objective function to maximise. We want to reward graphs that explain the data well, but we also want to appeal to Occam's razor (which we saw in Section 12.2.2) to ensure that the graphs are as simple as possible. Typical methods are to use an objective function based on the Minimum Description Length (MDL) (which is based on the argument that the solution with the shortest description, i.e., fewest parameters that explains the data, is the best one) or related information-theoretic measures. Then hill climbing or similar algorithms are used to perform local search around a set of random starting graphs. As usual for optimisation problems, getting the scoring function right is critical. You might be wondering why it is not possible to use a genetic algorithm. It is, but given the number of iterations of the GA,

each of which would involve constructing hundreds of possible networks, testing them by performing inference, and then combining them, the computational expense rules it out as a practical possibility. As it is such an important problem, there has been a lot of very advanced work on it, which is beyond our scope here. However, there are references at the end of the chapter that contain more information should you want it.

Given that we cannot make the entire graph, we will consider the compromise situation, where we try to compute the conditional probability tables for a known graph based on data. This is quite a sensible compromise: you assume that some expert can put together a network that shows how variables relate to each other, effectively a 'cartoon' of the data generating process, and then you use data in order to compute the conditional probability tables. However, it is still difficult. The idea is to choose the probability distributions to maximise the likelihood of the training data. If there are no hidden nodes, then it is possible to compute the likelihood directly:

$$\begin{aligned} L &= \frac{1}{M} \log \prod_{m=1}^{N} P(D_m|G) \\ &= \frac{1}{M} \sum_{n=1}^{N} \sum_{m=1}^{M} \log P(X_n|\text{parents}(X_n), D_m), \end{aligned} \tag{16.8}$$

where M is the number of training data examples D_m, and X_n is one of the N nodes in graph G. Equation (16.8) has broken everything into sums over each node individually, which means that we can compute each separate conditional probability table. To compute the values of the table, you just need to count how often you have been scared before an exam given each of the possible values for having revised and attended lectures, and normalise it to make it into a probability. The danger with this is that with small amounts of data there could be examples that have not happened in training, and that will therefore have probability 0, although this can be dealt with by including prior probabilities and using Bayes' rule to update the estimates using the real data.

Obviously, this doesn't work if there are hidden nodes, since we don't know values for them in the data. Surprisingly, getting around this problem isn't as difficult as might be expected. The key is to see that if we did have values for them, then Equation (16.8) could be used. We can estimate values for them by inference, and then we can iterate these two steps: an estimation step using inference followed by a maximisation step, making this an EM algorithm (Section 7.1.1).

There is lots more work on Bayesian networks, and the references at the end of the chapter include entire books on the topic for anybody wishing to explore more in this area. We will now turn our attention to some other types of graphical models, starting with the variation where the edges are undirected.

16.2 MARKOV RANDOM FIELDS

Bayesian networks are inherently asymmetric, since each edge had an arrow on it. If we remove this constraint, then there is no longer any idea of children and parent nodes. It also makes the idea of conditional independence that we saw for the Bayesian network easier: two nodes in a Markov Random Field (MRF) are conditionally independent of each other, given a third node, if there is no path between the two nodes that doesn't pass through the third node. This is actually a variation on the Markov property, which is how the networks got their name: the state of a particular node is a function only of the states of its immediate

neighbours, since all other nodes are conditionally independent given its neighbours. You might think that this fact would make inference on MRFs simpler, but unfortunately it doesn't; in general it is still a #P-hard problem. However, there are particular applications where MRF methods have turned out to be particularly useful, often for images.

The most well-known example is image denoising, something that we have already seen in Section 4.4.5 when we talked about auto-associative learning in the MLP. Suppose that we have a binary image I with pixel values $I_{x_i,x_j} \in \{-1, 1\}$. This image is a representation of an 'ideal' image I'_{x_i,x_j} that has no noise in it, which is what we want to recover. If we assume that the amount of noise is small, then there should be a good correlation between the values of each pixel in the two images, so I_{x_i,x_j} and I'_{x_i,x_j} should be correlated. We also assume that within a small 'patch' or region in an image, there is good correlation between pixels (so I_{x_i,x_j} should correlate well with I_{x_i+1,x_j} and its other neighbouring pixels (I_{x_i,x_j-1}, etc.). This assumption says that there are lots of places in the image where all of the pixels are of the same value, and this is (at least approximately) true for most images, and says that the pixels are correlated (and that other pixels in the image are conditionally independent of I_{x_i,x_j} given the neighbours of that pixel, which is the MRF bit).

The original theory of MRFs was worked out by physicists, initially by looking at the Ising model, which is a statistic description of a set of atoms connected in a chain, where each can spin up (+1) or down (-1) and whose spin affects those connected to it in the chain. Physicists tend to think of the energy of such systems, and argue that stable states are those with the lowest energy, since the system needs to get extra energy if it wants to move out of this state. For this reason, the jargon of MRFs is in terms of energies, and we therefore want the energy of our pair of images to be low when the pixels match, and higher when they do not. So we write the energy of the same pixel in two images as $-\eta I_{x_i,x_j} I'_{x_i,x_j}$, where η is a positive constant. Note that if the two pixels have the same sign then the energy is negative, while if they have opposite signs then the energy is positive and therefore larger. The energy of two neighbouring pixels is $-\zeta I_{x_i,x_j} I_{x_i+1,x_j}$, and we can just add these components together to get the total energy:

$$E(I, I') = -\zeta \sum_{i,j}^{N} I_{x_i,x_j} I_{x_i \pm 1, x_j \pm 1} - \eta \sum_{i,j=1}^{N} I_{x_i,x_j} I'_{x_i,x_j}, \tag{16.9}$$

where the index of the pixels is assumed to run from 1 to N in both the x and y directions in both images and we are only interested in locally flat patches of the image we are changing, which is I.

There is now a simple iterative update algorithm, which is to start with noisy image I and ideal I', and update I so that at each step the energy calculation is lower. So you pick one pixel I_{x_i,x_j} for some values of x_i, x_j at a time, and compute the energies with this pixel being set to -1 and 1, picking the lower one. In probabilistic terms, we are making the probability $p(I, I')$ higher. The algorithm then moves on to another pixel, either choosing a random pixel at each step or moving through them in some pre-determined order, running through the set of pixels until their values stop changing. Figure 16.5 shows an original black and white image, a version corrupted with 10% noise, and the MRF-reconstructed version using parameters $\eta = 2.0, \zeta = 1.5$. This reduces the error from 10% to less than 1%, although it also removes my home country of New Zealand from the map!

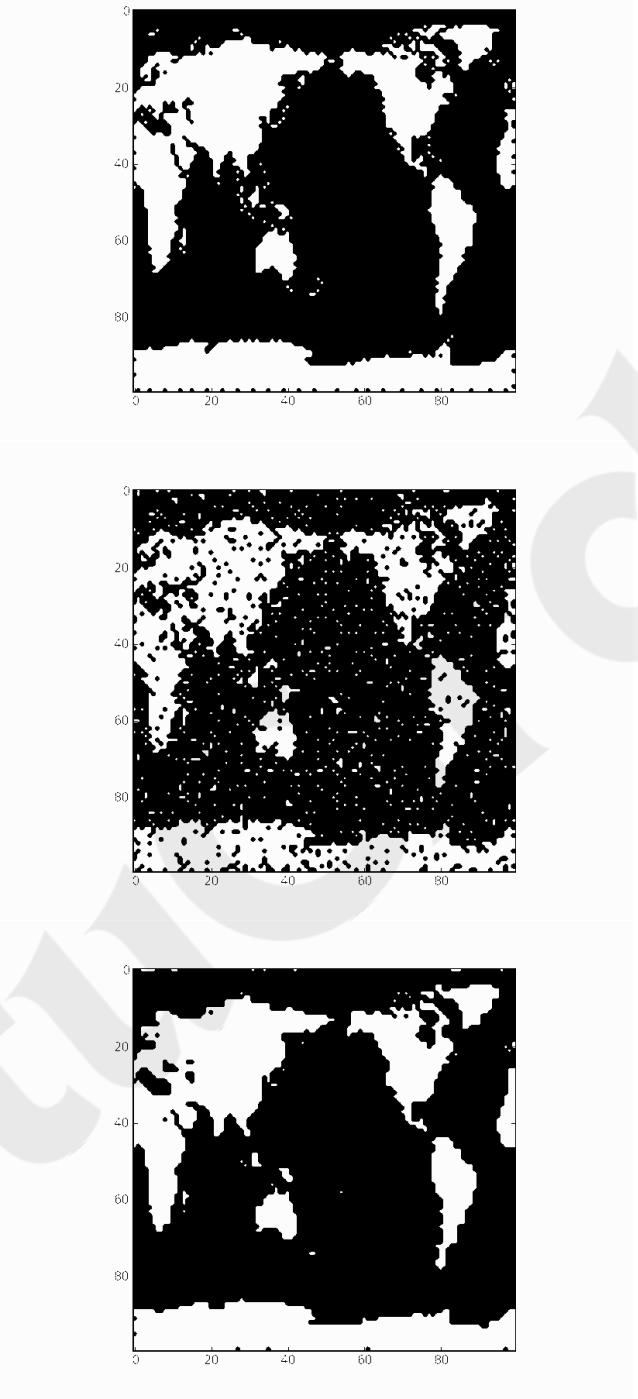


FIGURE 16.5 Using the MRF image denoising algorithm with $\eta = 2.1, \zeta = 1.5$ on a map of the world (*top left*) corrupted by 10% uniformly distributed random noise (*top right*) gives the image below which has about 1% error, although it has smoothed out the edges of all the continents.

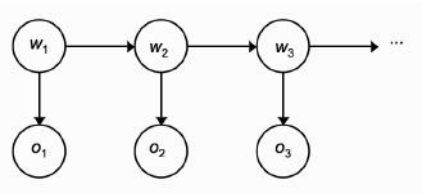


FIGURE 16.6 The Hidden Markov Model is an example of a dynamic Bayesian network. The figure shows the first three states and the related observations unrolled as time progresses.

The Markov Random Field Image Denoising Algorithm

- Given a noisy image I and an original image I', together with parameters η, ζ:
- Loop over the pixels of image I:
  - compute the energies with the current pixel being -1 and 1
  - pick the one with lower energy and set its value in I accordingly

We will now focus on a type of graphical model that is in very common use, and that has computationally tractable algorithms for doing exact inference on it.

16.3 HIDDEN MARKOV MODELS (HMMS)

The Hidden Markov Model is one of the most popular graphical models. It is used in speech processing and in a lot of statistical work. The HMM generally works on a set of temporal data. At each clock tick the system moves into a new state, which can be the same as the previous one. Its power comes from the fact that it deals with situations where you have a Markov model, but you do not know exactly which state of the Markov model you are in—instead, you see observations that do not uniquely identify the state. This is where the *hidden* in the title comes from. Performing inference on the HMM is not that computationally expensive, which is a big improvement over the more general Bayesian network. The applications that it is most commonly applied to are temporal: a set of measurements made at regular time intervals, which comprise the observations of the state. In fact, the HMM is the simplest dynamic Bayesian network, a Bayesian network that deals with sequential (often time-series) data. Figure 16.6 shows the HMM as a graphical model.

The example that we will use is this: As a caring teacher I want to know whether or not you are actually working towards the exam. I know from Chapter 12 that there are four things that you do in the evenings (go to the pub, watch TV, go to a party, study) and I want to work out whether or not you are studying. However, I can't just ask you, because you would probably lie to me. So all I can do is try to make observations about your behaviour and appearance. Specifically, I can probably work out if you look tired, hungover, scared, or fine. I want to use these observations to try to work out what you did last night. The problem is that I don't know why you look the way you do, but I can guess by assigning probabilities to those things. So if you look hungover, then I might give probability 0.5 to the guess that you went to the pub last night, 0.25 to the guess that you went to a party, 0.2 to watching TV, and 0.05 to studying. In fact, we will use these the other way round, using

the probability that you look hungover given what you did last night. These are known as observation or emission probabilities.

I don't have access to the other information that was used in Chapter 12, such as what parties are on and what other assignments you have (one of the worst things about stopping being a student is that the number of parties you get invited to drops off), but based on my own experience of being a student I can guess how likely parties are, etc., and knowing what student finances are, I can guess things like the probability of you going to the pub tonight if you went to the pub last night. So now it is just a question of putting these things into a form where I can work with them, and I can prepare my lectures according to how well you are working.

Each day that I see you in lectures I make an observation of your appearance, $o(t)$, and I want to use that observation to guess the state $\omega(t)$. This requires me to build up some kind of probabilities $P(o_k(t)|\omega_j(t))$, which is the probability that I see observation o_k (e.g., you are tired) given that you were in state ω_j (e.g., you went to a party) last night. These are usually labelled as $b_j(o_k)$. The other information that I have, or think I have, is the transition probability, which tells me how likely you are to be in state ω_j tonight given that you were in state ω_i last night. So if I think you were at the pub last night I will probably guess that the probability of you being there again tonight is small because your student loan won't be able to handle it. This is written as $P(\omega_j(t+1)|\omega_i(t))$ and is usually labelled as $a_{i,j}$.

I can add one more constraint to each of the probability distributions $a_{i,j}$ and b_i. I know that you did something last night, so $\sum_j a_{i,j} = 1$ and I know that I will make some observation (since if you aren't in the lecture I'll assume you were too tired), so $\sum_k b_j(o_k) = 1$. There is one other thing that is generally assumed, which is that the Markov chain is ergodic, something that we saw in Section 15.4.1: it means that there is a non-zero probability of reaching every state eventually, no matter what the starting state.

After a couple of weeks of the course I have made observations about you, and I am ready to sort out my HMM. There are three things that I might want to do with the data:

- see how well the sequence of observations that I've made match my current HMM (Section 16.3.1)
- work out the most probable sequence of states that you've been in based on my observations (Section 16.3.2)
- given several sets of observations (for example, by watching several students) generate a good HMM for the data (Section 16.3.3)

We will start by assuming that I invent a model and want to see how good it is. So I use my own knowledge of being a student to work out the probability distributions and then I can test the observations I make of you against my model. At this point I will probably find out that my student life was different to yours, or things have changed since I was a student, and I will have to generate a new model to match current data. I can then use this improved model to work out what you've been doing each evening. These problems are dealt with in the next three sections.

The HMM itself is made up of the transition probabilities $a_{i,j}$ and the observation probabilities $b_j(o_k)$, and the probability of starting in each of the states, π_i. So these are the things that I need to specify for myself, starting with the transition probabilities (which are also shown in Figure 16.7):

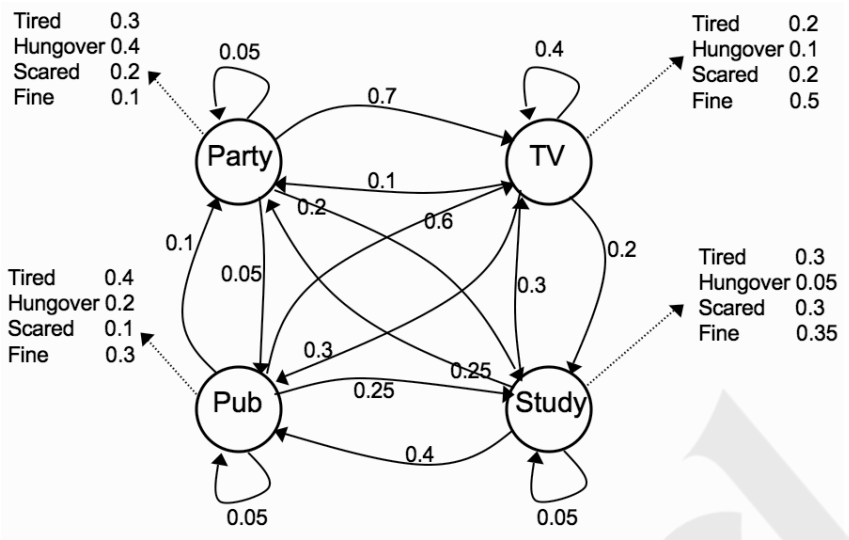


FIGURE 16.7 The example HMM with transition and observation probabilities shown.

	Previous night			
	TV	Pub	Party	Study
TV	0.4	0.6	0.7	0.3
Pub	0.3	0.05	0.05	0.4
Party	0.1	0.1	0.05	0.25
Study	0.2	0.25	0.2	0.05

and then the observation probabilities:

	TV	Pub	Party	Study
Tired	0.2	0.4	0.3	0.3
Hungover	0.1	0.2	0.4	0.05
Scared	0.2	0.1	0.2	0.3
Fine	0.5	0.3	0.1	0.35

16.3.1 The Forward Algorithm

Suppose that I see the following observations: $O =$ (tired, tired, fine, hungover, hungover, scared, hungover, fine) and I want to work out the likely run of states that generated it. The probability that my observations $O = \{o(1), \ldots, o(T)\}$ come from the model can be computed using simple conditional probability. I know you were doing something last night, so for an observation $o(t) =$tired (say) I just need to compute the probability that I made that observation given you were in a particular state (say watching TV) and multiply it by the probability that you were in that state given the state I thought you were in the night before (say partying). So for the example, I compute the probability that you were tired given that you were watching TV, which is 0.2, and then multiply it by the probability that you spent last night watching TV given that I thought you were partying the night before, which is 0.1. So this yields probability 0.02 for this particular state change. There is one extra thing that we need which is to decide which state you actually start in. I don't know this, so I assign probability 0.25 to each state.

Now I need to do this over every possible sequence of states to find out the most likely one based on what I actually saw. Note that I have used O to denote the whole sequence of observations that I made. In the same way, Ω is an entire sequence of possible states (this is a change in notation from the rest of the methods we have looked at, but it is consistent with the way that other authors describe HMMs). This can be written as:

$$P(O) = \sum_{r=1}^{R} P(O|\Omega_r)P(\Omega_r). \tag{16.10}$$

The r index here describes a possible sequence of states, so Ω_1 is one sequence, Ω_2 another, and so on. We'll consider this in a minute, but first we will use the Markov property to write:

$$P(\Omega_r) = \prod_{t=1}^{T} P(\omega_j(t)|\omega_i(t-1)) = \prod_{t=1}^{T} a_{i,j}, \tag{16.11}$$

and

$$P(O|\Omega_r) = \prod_{t=1}^{T} P(o_k(t)|\omega_j(t)) = \prod_{t=1}^{T} b_j(o_k). \tag{16.12}$$

So Equation (16.10) can be written as:

$$\begin{aligned} P(O) &= \sum_{r=1}^{R}\prod_{t=1}^{T} P(o_k(t)|\omega_j(t))P(\omega_j(t)|\omega_i(t-1)) \\ &= \sum_{r=1}^{R}\prod_{t=1}^{T} b_j(o_k)a_{i,j}. \end{aligned} \tag{16.13}$$

This looks fairly easy now. The only problem is in that sum over r, which runs over all possible sequences of hidden states. If there are N hidden states then there are N^T possible sequences, and for each one we have to compute a product of T probabilities. Not only will these probabilities be incredibly small, but the computational cost of getting them will be astronomical: $\mathcal{O}(N^T T)$.

Fortunately, the Markov property comes to our rescue again. Since the probability of each state only depends on the data at the current and previous timestep $(o(t), \omega(t), \omega(t-1))$ we can build up our computation of $P(O)$ one timestep at a time. This is known as the forward trellis by some people, since it looks like a garden trellis in Figure 16.8. To construct the trellis we introduce a new variable $\alpha_i(t)$ that describes the probability that at time t the state is ω_i and that the first $(t-1)$ steps all matched the observations $o(t)$:

$$\alpha_j(t) = \begin{cases} 0 & t = 0, j \neq \text{initial state} \\ 1 & t = 0, j = \text{initial state} \\ \sum_i \alpha_i(t-1)a_{i,j}b_j(o_t) & \text{otherwise.} \end{cases} \tag{16.14}$$

where $b_j(o_t)$ means the particular emission probability of output o_t. This ensures that only the observation probability that has the index that matches the observation o_t contributes to the sum. Computing $P(O)$ now requires only $\mathcal{O}(N^2T)$, which is a substantial improvement, in a very simple algorithm, as will be seen shortly.

We will use the following notation: $a_{i,j}$ is the transition probability of going from state i

to state j, so if there are N states, then it is of size $N \times N$; $b_i(o)$ is the transition probability of emitting observation o in state i, so it is of size $N \times O$, where O is the number of different observations that there are (four in the example). It will be useful to introduce four more variables, all of which are probabilities that are conditioned on the observation sequence and the model:

- $\alpha_{i,t}$, which is the probability of getting the observation sequence up to time t and being in state i at time t (size $N \times t$),
- $\beta_{i,t}$, which is the probability of the sequence from $t+1$ to the end given that the state is i at time t,
- $\delta_{i,t}$, which is the highest probability of any path that reaches state i at time t,
- $\xi_{i,j,t}$, which is the probability of being in state i at time t, state j at time $t+1$, and so is an $N \times N \times T$ matrix.

Since $\alpha_{i,t}$ is the probability of getting the observation sequence up to time t and being in state i at time t conditioned on the model and the observations, the probability of the whole observation sequence given the model is just $\sum_{i=1}^{N} \alpha_{i,T}$.

The HMM Forward Algorithm

- Initialise with $\alpha_{i,0} = \pi_i b_i(o_0)$
- For each observation in order o_t, $t = 1, \ldots, T$
  - for each of the N_s possible states s:
    - $\alpha_{s,t+1} = b_s(o_{t+1}) \left(\sum_{i=1}^{N} (\alpha_{i,t} a_{i,s}) \right)$

Let's look at the first two states of our example HMM. In both, the observation is that you were tired, so we need to compute $\alpha_{i,t}$ and so construct the trellis. Figure 16.8 shows the idea, with the initial $\alpha_{i,t=0}$ coming from my guesses about how likely each state is (the π variable), and we just need to run through the set of computations to compute $\alpha_{i,t=2}$ and so on, getting the numbers that are shown in the figure. We then repeat this for the next step and so on until we reach the final state. At this stage we can sum up all of the possible probabilities, which tells me in this case that you were most likely watching TV last night. We will also need to be able to go backwards through the trellis, which is a very similar algorithm that works backwards to compute β values, also based on the transmission probability and observation probability matrices by:

$$\beta_{i,t} = \sum_{j=1}^{N} a_{i,j} b_j(o_{t+1}) \beta_{j,t+1}. \tag{16.15}$$

16.3.2 The Viterbi Algorithm

The next problem that we want to solve is the decoding problem of working out the hidden states: I can use my model of how students are expected to behave and match them with my observations to guess what you have been doing each evening. The algorithm is known as the Viterbi algorithm after its creator, although he actually derived it for error correction, a

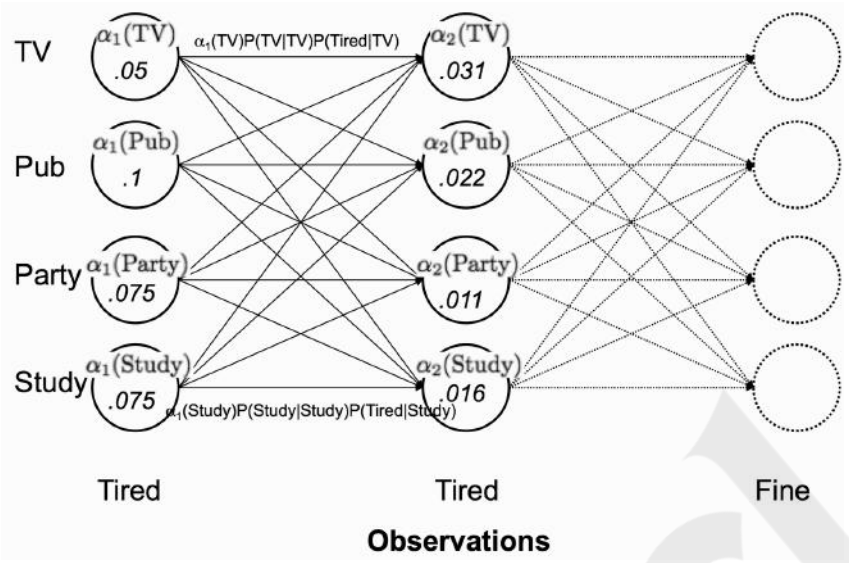


FIGURE 16.8 The forward trellis for the first two observations of the example HMM.

completely different application! We want to work out the $\delta_{i,t}$ variable. This requires finding the maximum probability of any path that gets us to state i at time t that has the right observations for the sequence up to t; apart from being a maximisation instead of a sum, it is pretty similar to the forward algorithm. The initialisation is $\delta_{i,t=0} = \pi_i b_i(o_0)$, since this tells us about the first observations we see, and then we work from there computing each new δ as:

$$\delta_{j,t+1} = \max_i \left(\delta_{i,t} a_{i,j}\right) b_j(o_{t+1}). \tag{16.16}$$

It will also be useful to keep track of which state seems to be the best at each stage: $\phi_{j,t} = \arg\max_i(\delta_{i,t-1}a_{i,j})$, because at the end of the sequence we want to go backwards through the matrix that we have built up and work out the actual most probable path from it.

So once we reach the end, we can work out the most likely state as the one with the highest $q_T^* = \delta\cdot, T$, and then work back through the lattice using $q_t^* = \phi_{q_{t+1}^*, t+1}$ until we reach the start of the sequence. As an algorithm this can be written as:

The HMM Viterbi Algorithm

- Start by initialising $\delta_{i,0}$ by $\pi_i bi(o_0)$ for each state i, $\phi_0 = 0$
  - run forward in time t:
    - for each possible state s:
      - $\delta_{s,t} = \max_i(\delta_{i,t-1}a_{i,s})b_s(o_t)$
      - $\phi_{s,t} = \arg\max_i(\delta_{i,t-1}a_{i,s})$
  - set q_T^*, the most likely end hidden state to be $q_T^* = \arg\max_i \delta_{i,T}$
  - run backwards in time computing:
    - $q_{t-1}^* = \phi_{q_t^*,t}$

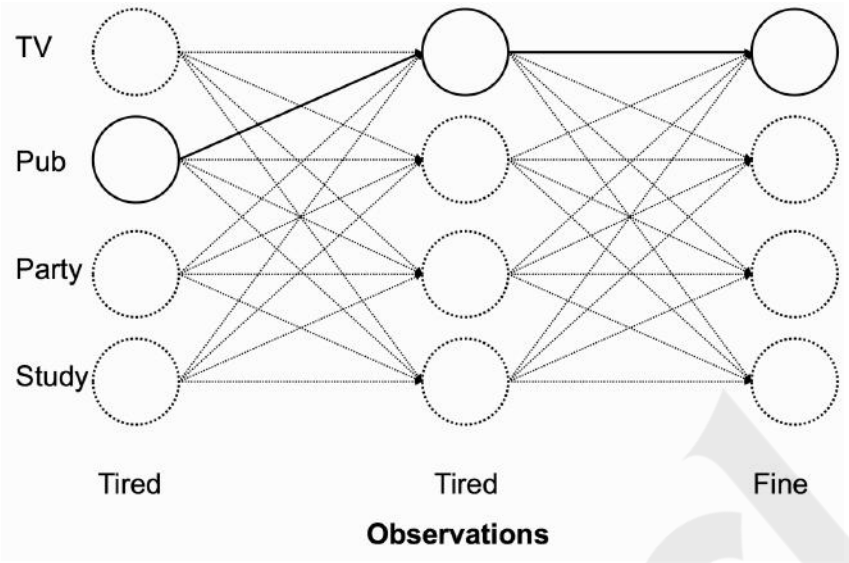


FIGURE 16.9 The Viterbi trellis for the first three observations of the example HMM.

Figure 16.9 shows the path for the first three states of the example. Using the numbers from the example, I can use the Viterbi algorithm to find the most likely explanation of a set of observations such as (fine, hungover, hungover, fine, tired, fine, fine, fine, hungover, hungover, tired, scared, scared), which tells me that you seem to have spent a lot of time in the pub. However, even for this most likely sequence, the probability of it is only 7.65×10^{-9}, which doesn't seem very likely. This is one of the problems with HMMs: the state space is so large that the probabilities tend to 0 very quickly. This is both an interpretation problem and a computational one, since we get problems with rounding errors very quickly because the probabilities are so small. This is discussed briefly at the end of this section.

16.3.3 The Baum–Welch or Forward–Backward Algorithm

In the example I had to invent the transition and observation probabilities from my experience, and the result is that the best path is not very likely. It would obviously be better to generate the HMM from sets of observations rather than by making up the transition probabilities. This is a learning process, and it is an unsupervised learning problem since we don't have any target solutions to go on. In fact, finding the optimal probabilities is an NP-complete problem, since we have to search over all the possible sets of probabilities for all the possible sequences. Instead, we will use an EM algorithm known as the Baum–Welch algorithm (see Section 7.1.1 for a previous example of an EM algorithm). This is not quite as good as the previous one in that it is not guaranteed to find even a local optima, but in general it works fairly well.

The key to the algorithm is in its second name: Forward–Backward. We introduced a variable α above that took us forward through the HMM above, and we mentioned that it had a complementary variable β that takes us backwards through the HMM, i.e., $\beta_i(t)$ tells us the probability that at time t we are in state ω_i and the result of the target sequence

(times $t+1$ to T) will be generated correctly. So we can now pick any point in the middle of a sequence, and run forwards from the beginning and backwards from the end to see the possible paths.

To see what computations are needed we will work out what the three variables that we are interesting in fitting—π_i, $a_{i,j}$ and $b_i(o_k)$—are, which are respectively the number of times we expect to be in state i at the first observation, the expected number of times we transition from state i to state j divided by the number of times we leave state i, and the expected number of times we see observation o_k when in state i, divided by the number of times we are in state i.

Thinking about the $\xi_{i,j,t}$ variable that we talked about, but haven't used yet, and which is the probability of being in state i at time t, state j at time $t+1$, we can see that (where the $\hat{\cdot}$ is to make it clear that these are estimates based on the sequences that we see):

$$\hat{\pi}_i = \sum_{j=1}^{N} \xi_{i,j,0} \tag{16.17}$$

$$\hat{a}_{i,j} = \sum_{t=1}^{T-1} \xi_{i,j,t} / \sum_{t=1}^{T-1} \sum_{j=1}^{N} \xi_{i,j,t} \tag{16.18}$$

$$\hat{b}_i(o_k) = \sum_{t=1, o_t=k}^{T} \sum_{j=1}^{N} \xi_{i,j,t} / \sum_{t=1}^{T} \sum_{j=1}^{N} \xi_{i,j,t} \tag{16.19}$$

The notation in the last line means that in the numerator we only include those times t where the observation k was seen, and note that the sums over time in the middle line only go up to $T-1$, since it isn't possible to move on from the last state.

So now the algorithm needs to start by computing $\xi_{i,j,t}$, and then make an estimate of π, a, b, which can then be iterated until the values stop changing. This has the flavour of an EM algorithm: we work out how many times we can expect to transition between states, which is the expectation, and then we try to maximise them.

The only thing that we haven't done yet is to work out how to compute $\xi_{i,j,t}$, although looking back at the definitions of the α and β variables, and working out that what we are doing is running forwards until time t, when we get to state i, then transitioning to state j carrying on from there to the end (or equivalently, going backwards from the end to state j at time $t+1$, we see that the form of $\xi_{i,j,t}$ is:

$$\xi_{i,j,t} = \frac{\alpha_{i,t} a_{i,j} b_j(o_{t+1}) \beta_{j,t+1}}{\sum_{i=1}^{N} \sum_{j=1}^{N} \alpha_{i,t} a_{i,j} b_j(o_{t+1}) \beta_{j,t+1}}, \tag{16.20}$$

where the numerator is simply a normaliser, and the values at T are a little different as there is no b or β then. This leads to the complete Baum–Welch algorithm:

The HMM Baum–Welch (Forward–Backward) Algorithm

- Initialise π to be equal probabilities for all states, and a, b randomly unless you have prior knowledge
- While updates have not converged:
  - **E-step:**
  - use forward and backward algorithms to get α and β
  - for each observation in the sequence $o_t, t = 1 \dots T$
    - for each state i:
      - for each state j:
      - compute ξ using Equation (16.20)
  - **M-step:**
  - for each state i:
    - compute $\hat{\pi}_i$ using Equation (16.17)
    - for each state j:
      - compute $\hat{a}_{i,j}$ using Equation (16.18)
  - for each different possible observation o:
    - compute $\hat{b}_i(o)$ using Equation (16.19)

Since this is the most important algorithm, here is a Python implementation as well:

```
def BaumWelch(obs,nStates):

        T = np.shape(obs)[0]
        xi = np.zeros((nStates,nStates,T))

        # Initialise pi, a, b randomly
        pi = 1./nStates*np.ones((nStates))
        a = np.random.rand(nStates,nStates)
        b = np.random.rand(nStates,np.max(obs)+1)

        tol = 1e-5
        error = tol+1
        maxits = 100
        nits = 0
        while ((error > tol) & (nits < maxits)):
                nits += 1
                oldpi = pi.copy()
                olda = a.copy()
                oldb = b.copy()

                # E step
                alpha,c = HMMfwd(pi,a,b,obs)
```

```
        beta = HMMbwd(a,b,obs,c)

        for t in range(T-1):
                for i in range(nStates):
                        for j in range(nStates):
                                xi[i,j,t] = alpha[i,t]*a[i,j]*b[j,
                                obs[t+1]]*beta[j,t+1]
                xi[:,:,t] /= np.sum(xi[:,:,t])

        # The last step has no b, beta in
        for i in range(nStates):
                for j in range(nStates):
                        xi[i,j,T-1] = alpha[i,T-1]*a[i,j]
        xi[:,:,T-1] /= np.sum(xi[:,:,T-1])

        # M step
        for i in range(nStates):
                pi[i] = np.sum(xi[i,:,0])
                for j in range(nStates):
                        a[i,j] = np.sum(xi[i,j,:T-1])/np.sum(xi[i,:,:
                        T-1])

                for k in range(max(obs)):
                        found = (obs==k).nonzero()
                        b[i,k] = np.sum(xi[i,:,found])/np.sum(xi[i,:,
                        :])

        error = (np.abs(a-olda)).max() + (np.abs(b-oldb)).max()
        print nits, error, 1./np.sum(1./c), np.sum(alpha[:,T-1])

    return pi, a, b
```

We can't really use this algorithm very well on the simple example, since we would need rather more data to do justice to the training. However, if we do apply it and then compute the Viterbi path, then it gives the same answer as with the invented data.

One final thing to mention with the HMM is that there are ways to deal with the fact that the probabilities get so small, which can cause round-off errors inside the computer. One approach is to renormalise the α values by dividing by the sum of the α values for each time step. If the same values are used in the β calculation as well, then they cancel out beautifully in the final calculations and things work very well. This is the c variable in the following code for the forward algorithm:

```
def HMMfwd(pi,a,b,obs):

    nStates = np.shape(b)[0]
    T = np.shape(obs)[0]
```

```
    alpha = np.zeros((nStates,T))
    alpha[:,0] = pi*b[:,obs[0]]

    for t in range(1,T):
        for s in range(nStates):
            alpha[s,t] = b[s,obs[t]] * np.sum(alpha[:,t-1] * a[:,s])

    c = np.ones((T))
    if scaling:
        for t in range(T):
            c[t] = np.sum(alpha[:,t])
            alpha[:,t] /= c[t]
    return alpha,c
```

That pretty much sums it up for the HMM. It is worth mentioning two limitations of it, which are that the probability distributions are not time dependent, and that the probabilities can get very small. The second of these problems is an implementation detail that needs careful monitoring, while the first can be dealt with by using more general graphical models, although with the additional computational costs that come with that.

16.4 TRACKING METHODS

We will now look at two methods of performing tracking. You perform tracking fairly easily, keeping tabs on where something is and how it is moving. This has an obvious evolutionary benefit, since keeping track of where predators were and whether they were coming towards you could keep you alive. It is also useful for a machine to be able to do this, both for similar reasons to a human or animal (watching something moving and predicting what path it will follow, for example in radar or other imaging method) and to keep track of a changing probability distribution. We will look at two methods of doing it, the Kalman filter and the particle filter.

16.4.1 The Kalman Filter

The Kalman filter (named for E. Kalman; although he was not the original inventor he did do quite a lot of work on it) is a recursive estimator. It makes an estimate of the next step, then computes an error term based on the value that was actually produced in the next step, and tries to correct it. It then uses both of those to make the next prediction, and iterates this procedure. It can be seen as a simple cycle of predict-correct behaviour, where the error at each step is used to improve the estimate at the next iteration. The Kalman filter can be represented by the graphical model shown in Figure 16.10.

Much of the jargon that is associated with the Kalman filter is familiar to us: the state, which is hidden, consists of the variables that we want to know, which we see through noisy observations over time. There is a transition model that tells us how states change from one to another, and an observation model (also called the sensor model here) that tells us how states lead to observations.

The underlying idea is that there is some time-varying process that is generating a set of noisy outputs, where there are two sources of noise: process noise, which represents

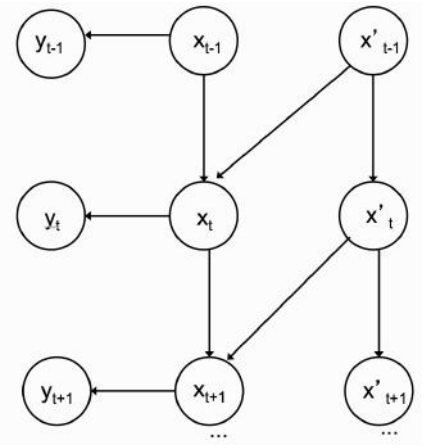


FIGURE 16.10 A representation of the Kalman filter with time derivatives (such as for tracking) as a graphical model.

the fact that the process changes over time, but we don't know how, and observation (or measurement) noise, which is the errors that are made in the readings. Both are assumed to be independent of each other, and zero mean Gaussians. We write the process as a stochastic difference equation in $\mathbf{x}$, which has n dimensions:

$$\mathbf{x}_{t+1} = \mathbf{A}\mathbf{x}_t + \mathbf{B}\mathbf{u}_t + \mathbf{w}_t, \tag{16.21}$$

where $\mathbf{A}_t$ is an $n \times n$ matrix that represents the non-driven part of the underlying process, $\mathbf{B}$ is an $n \times l$ matrix that represents the driving force, and $\mathbf{u}$ is the l-dimensional driving force. $\mathbf{w}$ is the process noise, which is assumed to be zero mean with standard deviation $\mathbf{Q}$.

The observations that we make are m-dimensional:

$$\mathbf{y}_t = \mathbf{H}\mathbf{x}_t + \mathbf{v}_t, \tag{16.22}$$

where $m \times n$ matrix $\mathbf{H}$ describes how measurements of the state are measured, and $\mathbf{v}$ is the measurement noise, which is also assumed to be zero mean, but with standard deviation $\mathbf{R}$.

As an example of this, consider a particle moving at a constant speed in one dimension. The state is two dimensional, consisting of the position and velocity of the particle (so $\mathbf{x} = [x, \dot{x}]^T$). Since there is no driving force the next term is $\mathbf{B}\mathbf{u}_t = 0$. The process equation is then derived by using Newton's laws of motion (so $x_{t+1} = x_t + \Delta t \dot{x}_t$, and since we are making indexing by time, $\Delta_t = 1$, and the velocity is constant):

$$\begin{pmatrix} x_{t+1} \\ \dot{x}_{t+1} \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} x_t \\ \dot{x}_t \end{pmatrix} + \mathbf{w}_t. \tag{16.23}$$

We can observe the position of the particle, up to measurement noise, but not the velocity, and so the measurement equation is:

$$y_t = \begin{pmatrix} 1 & 0 \end{pmatrix} \begin{pmatrix} x_t \\ \dot{x}_t \end{pmatrix} + \mathbf{v}_t. \tag{16.24}$$

The principal simplifying assumptions of the Kalman filter are that the process is linear and that all of the distributions are Gaussian with constant covariance. Since the convolution of Gaussians is also Gaussian, this means that we can put them together to form new

Gaussians, and so the model stays well behaved. This was a significant advantage over previous methods of tracking, which tended to stop working fairly quickly, since the estimates broke down because the probability distribution stopped being well-defined. We assume that both the transition model and the observation model are Gaussians with means based on the previous observations, and fixed covariances $\mathbf{Q}$ and $\mathbf{R}$ which can be written mathematically as (with the same parameters as the state and measurement update equations):

$$P(\mathbf{x}_{t+1}|\mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t+1}|\mathbf{A}\mathbf{x}_t, \mathbf{Q}) \quad (16.25)$$

$$P(\mathbf{z}_t|\mathbf{x}_t) = \mathcal{N}(\mathbf{z}_t|\mathbf{H}\mathbf{x}_t, \mathbf{R}). \quad (16.26)$$

Having described the set-up, what do we do? The basic idea is to make a prediction and then correct it when the next observation is available, i.e., at the next timestep. We will use $\hat{\mathbf{x}}$ and $\hat{\mathbf{y}}$ as the estimates, so $\hat{\mathbf{y}}_{t+1} = \mathbf{H}\mathbf{A}\hat{\mathbf{x}}_{t+1}$ and so the error is $\mathbf{y}_{t+1} - \hat{y}_{t+1}$; that is the difference between what was actually observed and what we predicted (without measurement noise). Since this is a probabilistic process with Gaussian distributions, we can also keep a predicted covariance matrix that goes with it: $\hat{\mathbf{\Sigma}}_{t+1} = \mathbf{A}\mathbf{\Sigma}_t\mathbf{A}^T + \mathbf{Q}$ (which is $E[(\mathbf{x}_k - \hat{\mathbf{x}}_k)(\mathbf{x}_k - \hat{\mathbf{x}}_k)^T]$). The Kalman filter weights these error computations by how much trust the filter currently has in its predictions; these weights are known as the Kalman gain and are computed by:

$$\mathbf{K}_{t+1} = \hat{\mathbf{\Sigma}}_{t+1}\mathbf{H}^T\left(\mathbf{H}\hat{\mathbf{\Sigma}}_{t+1}\mathbf{H}^T + \mathbf{R}\right)^{-1}. \quad (16.27)$$

This equation comes from minimising the mean-square error; we will not derive it, but the Further Reading section gives further references for you to follow up if you wish. Using it, the update for the estimate is:

$$\mathbf{x}_{t+1} = \hat{\mathbf{x}}_{t+1} + \mathbf{K}_{t+1}\left(\mathbf{z}_{t+1} - \mathbf{H}\hat{\mathbf{x}}_{t+1}\right), \quad (16.28)$$

All that is then required is to update the covariance estimate:

$$\mathbf{\Sigma}_{t+1} = (\mathbf{I} - \mathbf{K}_{t+1}\mathbf{H})\hat{\mathbf{\Sigma}}_{t+1}, \quad (16.29)$$

where $\mathbf{I}$ is the identity matrix of the relevant size. Putting these equations together leads to a simple algorithm.

The Kalman Filter Algorithm

- Given an initial estimate $\mathbf{x}(0)$
- For each timestep:
  - **predict the next step**
    - predict state as $\hat{\mathbf{x}}_{t+1} = \mathbf{A}\mathbf{x}_t + \mathbf{B}\mathbf{u}_t$
    - predict covariance as $\hat{\mathbf{\Sigma}}_{t+1} = \mathbf{A}\mathbf{\Sigma}_t\mathbf{A}^T + \mathbf{Q}$
  - **update the estimate**
    - compute the error in the estimate, $\boldsymbol{\epsilon} = \mathbf{y}_{t+1} - \mathbf{H}\mathbf{A}\mathbf{x}_{t+1}$
    - compute the Kalman gain using Equation (16.27)
    - update the state using Equation (16.28)
    - update the covariance using Equation (16.29)

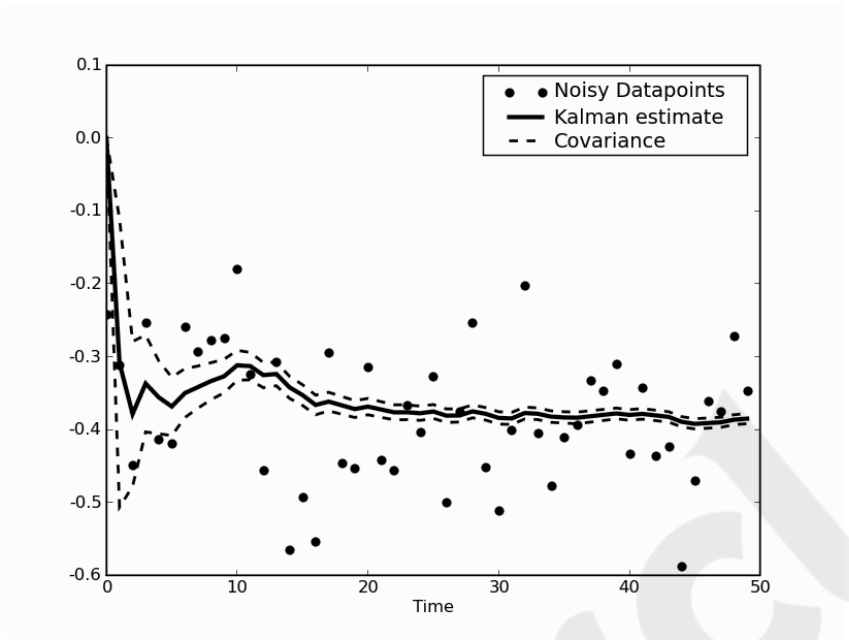


FIGURE 16.11 Estimates of a 1D constant noisy process using a Kalman filter. The filter settles to representing the unchanging mean of the process fairly quickly, and the estimated error (shown as dashed lines) drops accordingly.

Implementing this principally involves repeated use of `np.dot()` to multiply matrices together.

Figure 16.11 shows a simple 1D example of using the Kalman filter, where there is no time variation (so $x_{t+1} = x_t + w_k$). The dots are the noisy data from the process, and the line is the Kalman filter estimate, with the dashed lines representing one standard deviation. It can be seen that the initial estimate was not very good, but the algorithm quickly converges to a good estimate of the mean of the data, and the error drops accordingly.

Now that we have seen the Kalman filter in action, we need to work out how to use it for tracking. We worked out the process in 1D in Equations (16.23) and (16.24), and Figure 16.12 shows an example of 1D tracking, but we will write them in 2D here, to make sure that everything is clear. Again. we will assume that there is no control of the particle, so that it moves at constant velocity (up to noise).

The state of the particle is then:

$$\mathbf{x}_t = (x_1, x2, \dot{x}_1, \dot{x}_2)^T, \ \mathbf{y}_t = (y_1, y_2)^T, \tag{16.30}$$

$$\mathbf{A} = \begin{pmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}, \ \mathbf{H} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix} \tag{16.31}$$

In the absence of any other knowledge, we can assume that $\mathbf{Q}$ and $\mathbf{R}$ are proportional to the 4×4 and 2×2 identity matrices, respectively.

Figure 16.13 shows an example of a point moving in two spatial dimensions, starting at (10,10) and moving to the right at speed 1 for 15 steps, with noise standard deviation 0.1 in

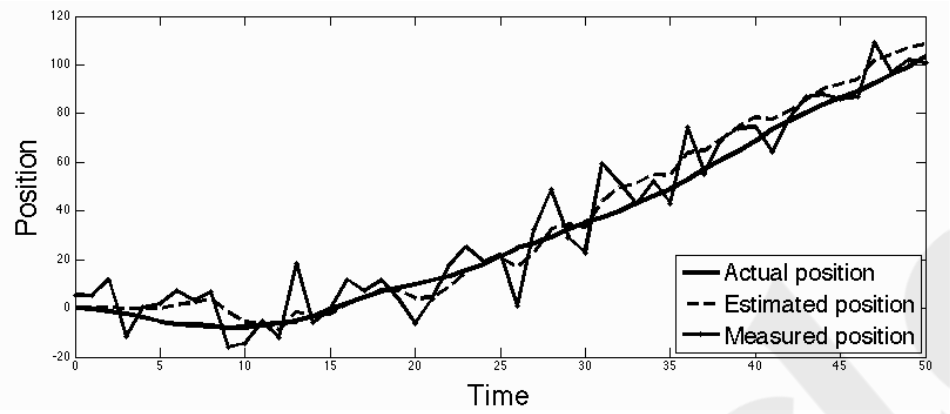


FIGURE 16.12 Demonstration of the Kalman filter tracking an object moving in one spatial dimension.

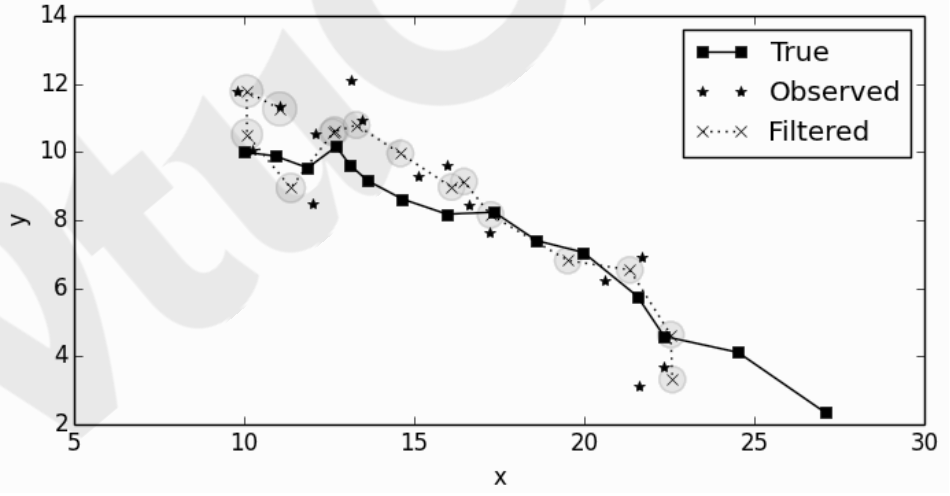


FIGURE 16.13 Demonstration of the Kalman filter tracking an object moving in two spatial dimensions. The grey circles show the covariance around each estimated point.

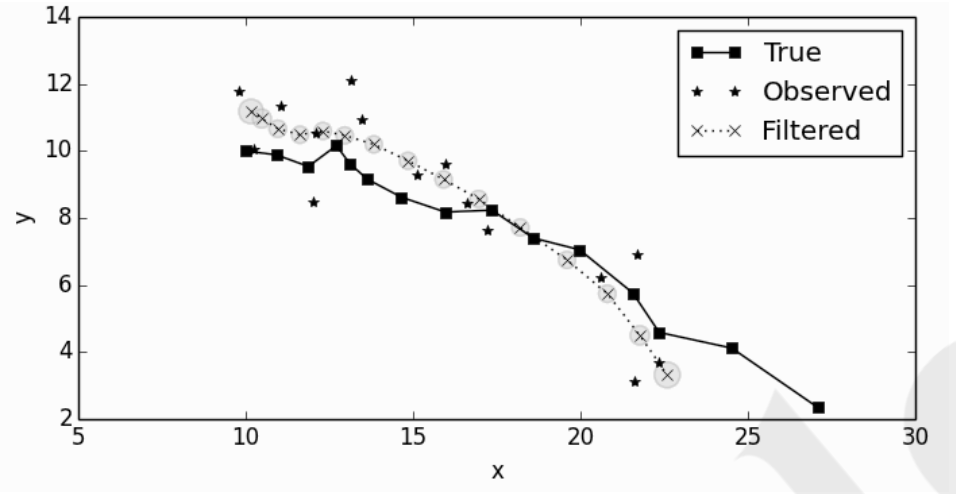


FIGURE 16.14 A smoothed version of the path from Figure 16.13.

both x_1 and x_2, with the grey circles representing the covariance matrix (at one standard deviation). It can be seen that the filter initially tracks the observations, but then learns more of the underlying process. However, the trajectory that is shown is rather 'jumpy' as the high level of noise affects the estimates. One way around this is to use the Kalman smoother, which performs a backwards smoothing of the trajectory after the filter has been used to estimate the positions. So the filter is run to predict the points, and then the predictions are updated to be a smoother path along the trajectory of the particle.

There are a variety of ways to do this smoothing, but one option, known as the Rauch–Tung–Striebel smoother is to use the following update equations from the endpoint back to the beginning (where the $\hat{(\cdot)}$ variables are the filtered versions):

$$\begin{aligned}
\mathbf{x}' &= \mathbf{A}\hat{\mathbf{x}} \\
\boldsymbol{\Sigma}' &= \mathbf{A}\hat{\boldsymbol{\Sigma}}\mathbf{A}^T + \mathbf{Q} \\
\mathbf{J} &= \hat{\boldsymbol{\Sigma}}\mathbf{A}\boldsymbol{\Sigma}' \\
\mathbf{x}_s &= \hat{\mathbf{x}} + \mathbf{J}(\hat{\mathbf{x}} - \mathbf{x}') \\
\boldsymbol{\Sigma}_s &= \hat{\boldsymbol{\Sigma}} + \mathbf{J}(\hat{\boldsymbol{\Sigma}} - \boldsymbol{\Sigma}')\mathbf{J}^T
\end{aligned} \tag{16.32}$$

Figure 16.14 shows the smoothed trajectory from Figure 16.13.

One of the main assumptions of the Kalman filter was that the process was linear. There are many cases where this is not true. One option to deal with non-linearity is to linearise about the current estimate $(\mathbf{x}_t, \boldsymbol{\Sigma}_t)$ and this leads to the Extended Kalman Filter. There are a lot of similarities with the original Kalman filter, so let's try and pick out the differences.

We start with some non-linear stochastic difference equation:

$$\mathbf{x}_{t+1} = f(\mathbf{x}_t, \mathbf{u}_t, \mathbf{w}_t), \tag{16.33}$$

where the variables are the same as for the Kalman filter, except that we have a non-linear function $f(\cdot)$ in place of the nice linear matrix, and we also have a measurement function:

$$\mathbf{y}_t = h(\mathbf{x}_t, \mathbf{v}_t). \tag{16.34}$$

If we have a current estimate $\hat{\mathbf{x}}_t$ then we can evaluate the function at that point by computing $\tilde{\mathbf{x}} = f(\hat{\mathbf{x}}, \mathbf{u}_t, 0)$, where we are assuming that the mean of the noise is 0. We now linearise about this point as:

$$x_{t+1} \approx \tilde{\mathbf{x}}_{t+1} + \mathbf{J}_{f,x}(\hat{\mathbf{x}}_t, \mathbf{u}_t, 0)(\mathbf{x}_t - \hat{\mathbf{x}}_t) + \mathbf{J}_{f,w}(\hat{\mathbf{x}}_t, \mathbf{u}_t, 0)\mathbf{w}_t), \tag{16.35}$$

and similarly for $h(\cdot)$ to get:

$$y_t \approx \tilde{\mathbf{y}}_{t+1} + \mathbf{J}_{h,x}(\tilde{\mathbf{x}}_t, 0)(\mathbf{x}_t - \tilde{\mathbf{x}}_t) + \mathbf{J}_{h,v}(\tilde{\mathbf{x}}_t, \mathbf{v}_t). \tag{16.36}$$

In both of these equations $\mathbf{J}$ refers to the Jacobian of the subscripted function with regard to the variable in the second subscript, so:

$$\mathbf{J}_{f,x}|_{i,j} = \frac{\partial f_i}{\partial \mathbf{x}_j}(\hat{\mathbf{x}}_t, \mathbf{u}_t, 0). \tag{16.37}$$

So providing that we can compute these two functions and their derivatives, we can use them to make an estimate of the error, which is a linear function and so can be estimated using the normal Kalman filter. This leads to the following algorithm:

The Extended Kalman Filter Algorithm

- Given an initial estimate $\mathbf{x}(0)$
- For each timestep:
  - **predict the next step**
    - predict state as $\hat{\mathbf{x}}_{t+1} = f(\hat{\mathbf{x}}_t, \mathbf{u}_t, 0$
    - compute the Jacobians $\mathbf{J}_{f,x}$ and $\mathbf{J}_{f,w}$
    - predict covariance as $\hat{\mathbf{\Sigma}}_{t+1} = \mathbf{J}_{f,x}\mathbf{\Sigma}_t\mathbf{J}_{f,x}^T + \mathbf{J}_{f,w}\mathbf{Q}\mathbf{J}_{f,w}^T$
  - **update the estimate**
    - compute the error in the estimate, $\boldsymbol{\epsilon} = \mathbf{y}_t - h(\hat{\mathbf{x}}_t, 0)$
    - compute the Jacobians $\mathbf{J}_{h,x}$ and $\mathbf{J}_{h,w}$
    - compute the Kalman gain using $\mathbf{K} = \mathbf{J}_{f,x}\mathbf{J}_{h,x}^T(\mathbf{J}_{h,x}\mathbf{J}_{f,x}\mathbf{J}_{h,x}^T + \mathbf{J}_{h,w}\mathbf{R}\mathbf{J}_{h,w}^T)^{-1}$
    - update the state using $\hat{\mathbf{x}} = \hat{\mathbf{x}} + \mathbf{K}\boldsymbol{\epsilon}$
    - update the covariance using $(\mathbf{I} - \mathbf{K}\mathbf{J}_{h,x})\mathbf{J}_{f,x}$

Figure 16.15 shows an example of the extended Kalman filter tracking the function $h(x, y, z) = x + y$ with $f(x, y, z) = (y, z, -.5x(y + z))$.

The Extended Kalman Filter is not an optimal estimator. Further, if the assumptions about local linearity that underlie the linearisation are not true, then the estimate is very poor, and even where it is good, it requires the computation of the Jacobians, which is potentially difficult. There have been various attempts to improve upon it; one method that may be of interest is to choose a set of points that represent the statistics of the data and to transform them by passing them through the non-linear functions ($f(\cdot)$ and $h(\cdot)$), and then to compute the statistics of those points in order to estimate the statistics of the transformed data. This has the great name of the unscented transform, and can be used to produce an Unscented Kalman Filter. For more information on this, see the Further Reading section; instead we will look at a common MCMC algorithm for performing tracking, the particle filter.

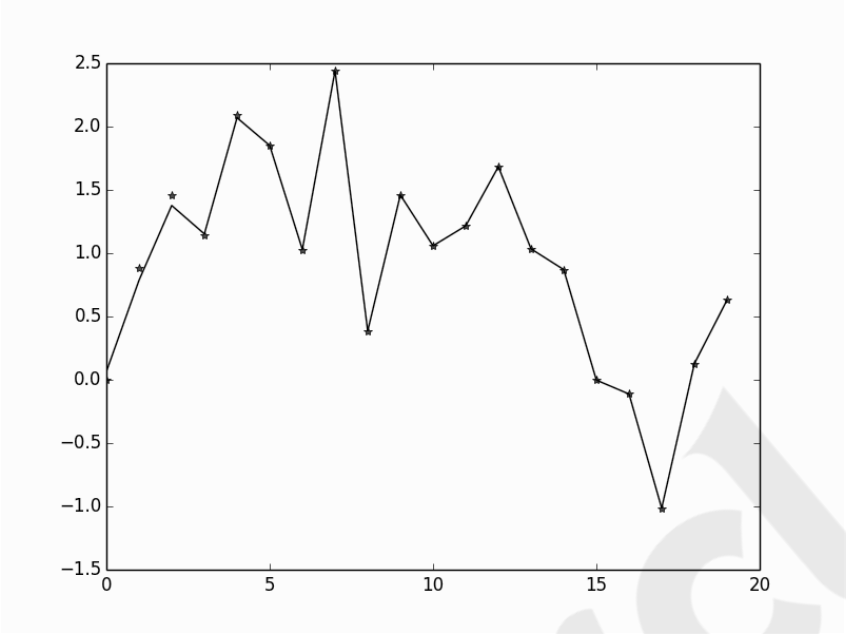


FIGURE 16.15 The extended Kalman filter tracking the function $h(x, y, z) = x + y$ with $f(x, y, z) = (y, z, -.5x(y + z))$.

16.4.2 The Particle Filter

As well as the linearity of the function, the Kalman filter also assumes that the distributions are Gaussian, so that they can be convolved and stay as Gaussians. In order to get around this problem, we return to the methods that have underpinned many of the algorithms in this chapter: sampling. The particular sampling technique that we will use is the sampling-importance-resampling algorithm of Section 15.3, which forms the basis of the particle filter, or condensation method. This is a relatively recent development, and has been finding many successful applications in tracking, including in image and signal analysis. The idea is to use sampling to keep track of the state of the probability distribution. This is known as sequential sampling, since we are using a set of samples for time t to estimate the process at time $t + 1$, and then resampling from there.

One benefit of sampling methods is that we don't have to hold on to the Markov assumption. In tracking, prior history can be useful, which means that the Markov assumption is a bad one. The proposal distribution is generally written as $q(\mathbf{x}_{t+1}|\mathbf{x}_{0:t}, \mathbf{y}_{0:t})$ to make this dependence clear, and the proposal distribution that is generally used in the estimated transition probabilities $p(\hat{\mathbf{x}}_{t+1}|\mathbf{x}_{0:t}, \mathbf{y}_{01:t})$, since it is a simple distribution that is related to the process. With this decided, there is very little else to the basic particle filter: a schematic of one iteration of the particle filter is shown in Figure 16.16. The basic algorithm is given next, followed by some points about the implementation and some examples.

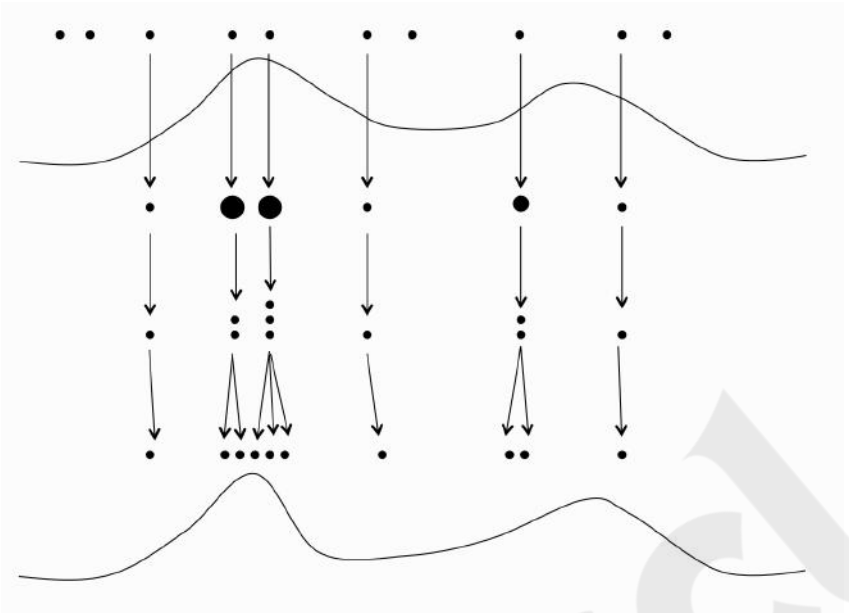


FIGURE 16.16 A schematic of one iteration of a particle filter. A set of random particle positioned have importance weights computed according to the distribution, and then new particles are created and modified based on these weights, and the estimated distribution is updated.

The Particle Filter Algorithm

- Sample $\mathbf{x}_0^{(i)}$ from $p(\mathbf{x}_0$ for $i = 1 \dots N$
- For each timestep:
  - **importance sample**
  - for each datapoint:
    - sample $\hat{\mathbf{x}}_t^{(i)}$ from $q(\mathbf{x}_t^{(i)} | \mathbf{x}_{0:t-1}^{(i)}, \mathbf{y}_{1:t})$
    - add $\hat{\mathbf{x}}_t^{(i)}$ onto the list of samples to get $\mathbf{x}_{0:t}^{(i)}$ from $\mathbf{x}_{0:t-1}^{(i)}$
    - compute the importance weights:

$$w_t^{(i)} = w_{t-1}^{(i)} \frac{p(\mathbf{y}_t | \hat{\mathbf{x}}_t^{(i)}) p(\mathbf{x}_t^{(i)} | \hat{\mathbf{x}}_{t-1}^{(i)})}{q(\mathbf{x}_t^{(i)} | \mathbf{x}_{0:t-1}^{(i)}, \mathbf{y}_{1:t})} \tag{16.38}$$

  - normalise the importance weights by dividing by their sum
  - **resample the particles**
    - retain particles according to their importance weights, so that there might be several copies of some particles, and none of others to get the same number of particles approximately sampled from $p(\mathbf{x}_{0:t}^{(i)} | \mathbf{y}_{1:t})$

Vincircle

The resampling part of this algorithm deserves a little more consideration, because there are several ways in which it can be done, and they differ in their computational time and the variance of the estimates. The code on the website provides two implementations: systematic resampling and residual resampling. The basic idea of systematic resampling is to use the cumulative sum of the weights (which will be 1 at the very end by definition) and a set of uniform random numbers $\tilde{u}_k$, put together so that they are in order using $u_k = (k = 1 + \tilde{u_k})/N$ and put n_i copies of particles i into the next set, where n_i is the number of u_k for which $\sum_{s=1}^{i-1} w_s \leq u_k < \sum_{s=1}^{i} w_s$. One way to implement this is using the following code:

```
def systematic(w,N):
    # Systematic resampling
    N2 = np.shape(w)[0]
    # One too many to make sure it is >1
    samples = np.random.rand(N+1)
    indices = np.arange(N+1)
    u = (samples+indices)/(N+1)
    cumw = np.cumsum(w)
    keep = np.zeros((N))
    # ni copies of particle xi where ni = number of u between ws[i-1] and ws[
    i]
    j = 0
    for i in range(N2):
        while((u[j]<cumw[i]) & (j<N)):
            keep[j] = i
            j+=1

    return keep
```

Residual sampling tries to speed this up (although it is an $\mathcal{O}(n)$ algorithm, it is called often, and so making it fast is a very good idea) by first using the integer part of Nw_i as an idea of how many copies of each particle i to keep, and then using stratified samples for the rest.

Figure 16.17 shows the particle filter keeping track of a distribution that changes at $t = 30$. The positions of the particles are shown as dots, the underlying state process is shown as the dashed line, and the observations are shown as pluses. The solid line is the hypothesised observation based on the mean of the particles. It can be seen that this tracks the observations very well.

As an example of using the Particle Filter for tracking, Figure 16.18 shows an object moving at constant speed being tracked in 2D. The weights are computed based on the Euclidean distance between each particle and the object, and the particles are resampled if the average distance between them grows too large.

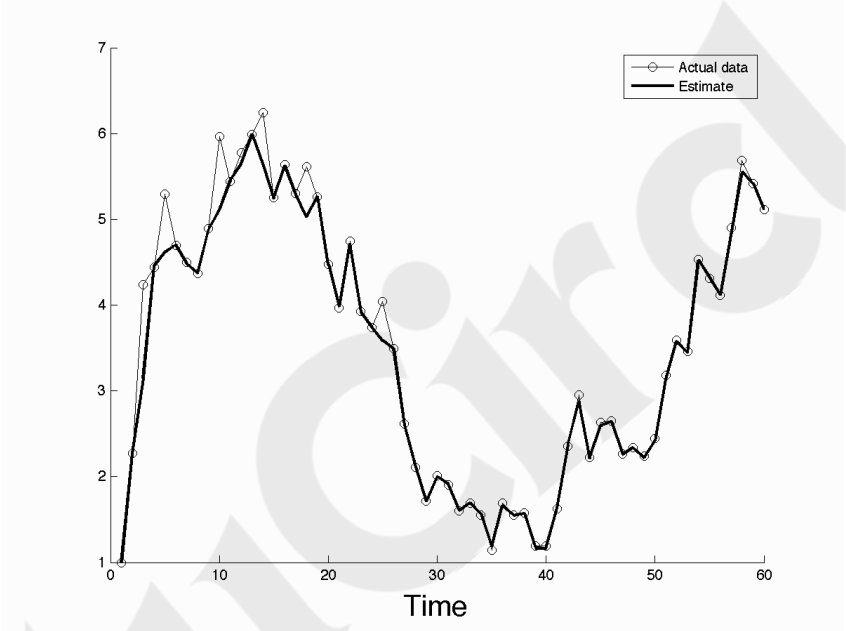


FIGURE 16.17 A particle filter tracking a 1D distribution that changes at $t = 30$. The observations are marked as pluses, the underlying state is shown by the dashed line, the particles at each iteration are shown as dots, and the hypothesised observation based on the mean of the particles is shown as a solid line.

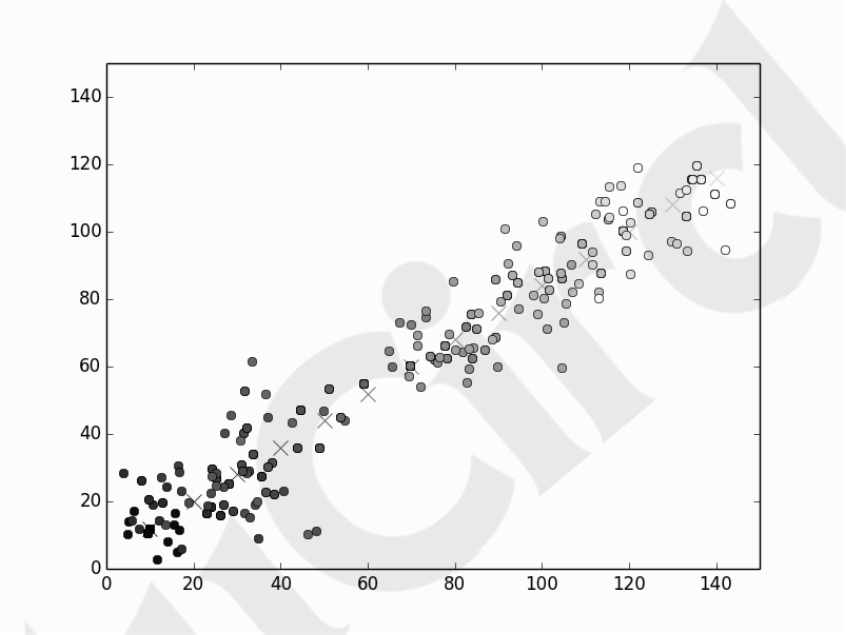


FIGURE 16.18 A particle filter tracking an object moving at a constant speed upwards and to the right in 2D. The crosses show the position of the object, and the circles show 15 particles at each iteration, starting at black ($t = 0$) and fading to white ($t = 10$). It can be seen that the particles track the object successfully.